

RecoMail AI

Personalized Retail Outreach Generator

1. Project Overview

RecoMail AI is an AI-powered tool that helps retail businesses send personalized emails to customers. Instead of sending the same email to everyone, the system creates unique emails for each customer based on their purchase history and spending level. Businesses can generate professional emails in seconds by uploading a customer file.

2. The Problem

Many businesses face these challenges:

- Customers ignore generic emails
- Low email open rates
- Wasted marketing time and money
- Customers expect personalized communication

3. Project Objectives

RecoMail AI is designed to solve real marketing problems by providing:

- Fast automation
- Better personalization
- Higher productivity
- Improved customer communication
- Smarter sales outreach

4. The Solution

RecoMail AI automatically creates personalized emails for each customer. Upload customer data, and the AI generates custom subject lines and email content based on what each customer bought before.

5. Key Features

- Simple CSV file upload
- Automatic data validation
- AI-generated personalized subject lines
- Custom email body for each customer

- Customer segmentation (Budget / Mid / Premium)
- Edit emails before sending
- Download results as CSV

6. Technologies Used

- Python - Programming language
- Streamlit - User interface
- Google Gemini 1.5 Flash - AI for email generation
- Pandas - Data processing
- CSV - Customer data format

7. How It Works

- Step 1: Upload customer CSV file
- Step 2: System validates the data
- Step 3: AI generates personalized emails
- Step 4: Preview emails on screen
- Step 5: Edit if needed
- Step 6: Download results

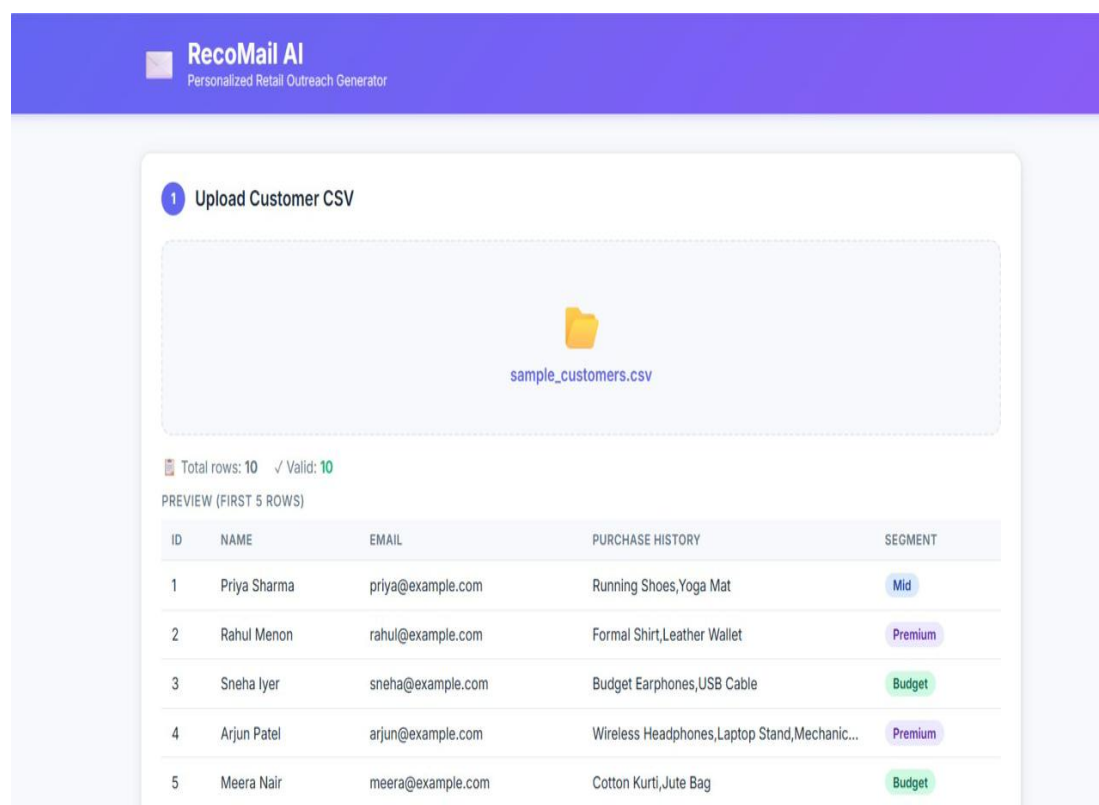
8. Application Screenshots

Screenshot 1: Email Preview Display

#4	Arjun Patel arjun@example.com	♥ Premium	⋮ Edit	▼ Expand
Subject: An Exclusive Selection for You, Arjun				
#5	Meera Nair meera@example.com	♥ Budget	⋮ Edit	▼ Expand
Subject: More great finds for you, Meera!				
#6	Vikram Singh vikram@example.com	♥ Mid	⋮ Edit	▼ Expand
Subject: Vikram, Elevate Your Fitness Journey!				
#7	Ananya Reddy ananya@example.com	♥ Premium	⋮ Edit	▼ Expand
Subject: Ananya, Elevate Your Routine				
#8	Kiran Joshi kiran@example.com	♥ Budget	⋮ Edit	▼ Expand
Subject: Kiran, More Value for Your Budget!				
#9	Divya Krishnan divya@example.com	♥ Mid	⋮ Edit	▼ Expand
Subject: New arrivals for your casual style				

This shows the email preview interface where users can see generated emails for each customer with their segment, subject lines, and options to edit or expand the full content.

Screenshot 2: CSV Upload & Data Validation



The upload section shows customer CSV file with automatic validation. It displays total rows, valid records, and a preview of the first 5 customer records with their names, emails, purchase history, and segments.

Screenshot 3: Upload & Generation Interface

**RecoMail AI**
Personalized Retail Outreach Generator

1 Upload Customer CSV



Drop your CSV here or click to browse

Required columns: customer_id, name, email, purchase_history, segment

2 Generate Emails

LLM Provider

Google Gemini (1.5 Flash) 



Shows the main interface with CSV upload area and email generation section. Users can select the AI model (Google Gemini 1.5 Flash) and click the Generate Emails button to start creating personalized emails.

Screenshot 4: Results & Download

2

Generate Emails

LLM Provider

Google Gemini (1.5 Flash) ▾

🔥 Generate Emails

✓ 10 succeeded

3

Preview & Download

Download CSV

10 emails generated

#1

Priya Sharma
priya@example.com

Mid

Edit

Expand

Subject: Elevate Your Fitness Routine, Priya!

#2

Rahul Menon
rahul@example.com

Premium

Edit

Expand

Subject: A Refined Addition to Your Collection

#3

Sneha Iyer
sneha@example.com

Budget

Edit

Expand

Subject: Great Tech Deals Just For You, Sneha!

After generation, users can preview all 10 generated emails with personalized subject lines. The interface shows success status and a Download CSV button to save results. Users can also edit individual emails before downloading.

9. Real-World Examples

Fashion Store

A clothing brand sends personalized emails recommending shoes, shirts, or accessories based on what each customer bought before.

Electronics Store

A gadget store promotes headphones, laptops, or chargers based on customer purchase history.

Fitness Brand

A sports company sends personalized offers for yoga mats, gym wear, and fitness products.

10. Advantages

- Easy to use - no technical knowledge needed
- Saves time - generates emails in seconds
- Better customer targeting
- Improves email engagement
- Professional email quality
- Works for businesses of any size

11. Limitations

- Requires internet connection
- Works best with clean customer data
- AI may need minor edits sometimes
- API costs for large-scale usage

12. Who Can Use This

- Retail store owners
- Online shopping businesses
- Marketing teams
- Startups and small businesses
- E-commerce companies

13. Technical Architecture

The system follows a step-by-step process from customer data to final emails:

- Customer CSV Upload
- Data Validation
- Customer Segmentation
- Prompt Generation
- Gemini AI Model Processing
- Email Generation
- Preview, Edit & Download

This architecture ensures smooth data flow from customer upload to final email generation.

14. Component Interaction

Each component works together to complete the email generation process:

- User - Initiates the process
- Frontend Interface - User-friendly web interface
- CSV Upload Module - Handles file upload
- Data Processing Engine - Processes and validates data
- Gemini AI API - Generates email content
- Generated Emails - AI-created personalized emails
- Download Output - Final results available for download

15. Pre-requirements

Before running the project, the following are needed:

- Python installed in system
- Internet connection
- Gemini API key
- VS Code / Jupyter / Any IDE
- Customer CSV file

16. Libraries & Dependencies

Main libraries used:

- streamlit

- pandas
- google-generativeai
- python-dotenv
- csv
- os
- json

Installation Command: `pip install streamlit pandas google-generativeai python-dotenv`

17. Hardware Requirements

Minimum Requirements

- Intel i3 Processor
- 4 GB RAM
- Basic Storage Space
- Internet Access

Recommended Requirements

- Intel i5 / Ryzen 5
- 8 GB RAM
- SSD Storage
- Stable Internet

18. Prior Knowledge Required

To understand this project, knowledge of the following is helpful:

- Python basics
- CSV file handling
- APIs
- Prompt engineering
- Streamlit basics
- Basic AI concepts

19. Project Workflow

- Step 1: Upload customer CSV file

- Step 2: Validate records
- Step 3: Choose AI model
- Step 4: Generate personalized emails
- Step 5: Preview output
- Step 6: Edit if needed
- Step 7: Download final CSV

20. Project Milestones

The RecoMail AI project was developed following a structured 7-milestone approach to ensure quality and systematic progression from concept to deployment.

Milestone 1 - Requirement Analysis

Phase Duration: Week 1-2

Understanding business needs and customer communication challenges. This phase involved:

- Identifying pain points in current email marketing processes
- Analyzing market demand for personalized email solutions
- Defining project scope and objectives
- Gathering stakeholder requirements

Milestone 2 - System Design & Environment Setup

Phase Duration: Week 3-4

This milestone focused on planning modules, designing the system interface, defining the overall workflow and architecture, and preparing the complete development environment. The team undertook systematic groundwork to ensure that all subsequent development activities would proceed smoothly, with a stable foundation in place from the very beginning. Activities included creating system architecture diagrams, designing the database schema for customer data, planning the React/Vite UI layout, and defining the API integration approach with the chosen LLM providers.

Activity 2.1 - Development Environment Setup

Setting up a clean and consistent development environment was the first critical step in Milestone 2. The team configured VS Code as the primary Integrated Development Environment (IDE) due to its robust support for Python, JavaScript, and React development, along with its

extensive extension ecosystem for linting, formatting, and debugging. Python virtual environments were created using the venv module to isolate project dependencies and prevent conflicts with system-level packages. The backend server was launched using Uvicorn, an ASGI server, which provided hot-reload capabilities during active development. This allowed developers to instantly see the effect of code changes without manually restarting the server. The frontend was bootstrapped using Vite with the React template, which offered an extremely fast development server with Hot Module Replacement (HMR).

The screenshot below captures the terminal output at the moment the Uvicorn server was launched successfully. The server began watching for file changes in the backend directory and confirmed it was running at `http://127.0.0.1:8000`. The reloader process (PID 4112) using WatchFiles was also started, enabling live code reload during development.

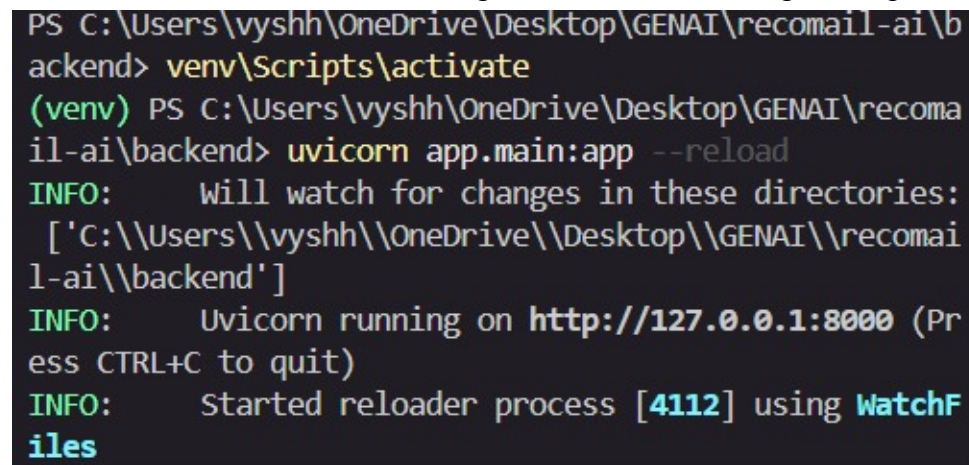
A terminal window with a dark background and light-colored text. The text shows the command prompt 'PS C:\Users\vyshh\OneDrive\Desktop\GENAI\recomail-ai\backend' followed by 'venv\Scripts\activate' and 'uvicorn app.main:app --reload'. The output shows 'INFO: will watch for changes in these directories: ['C:\Users\vyshh\OneDrive\Desktop\GENAI\recomail-ai\backend']', 'INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)', and 'INFO: Started reloader process [4112] using WatchFiles'.

Figure 2.1: Uvicorn development server running successfully at `http://127.0.0.1:8000` with WatchFiles hot-reload active

Activity 2.2 - Dependency Installation

All required software packages and libraries for both the backend and frontend components were identified, listed, and installed in this activity. For the Python backend, dependencies were managed via a `requirements.txt` file. Key packages installed included FastAPI as the web framework, Uvicorn as the ASGI server, python-dotenv for environment variable management, and various LLM client libraries for OpenAI, Google Gemini (`google-generativeai`), and Groq. The command `pip install -r requirements.txt` was used to install all backend dependencies in a single step within the activated virtual environment.

For the React frontend, Node.js and npm were used as the package manager. The `package.json` file declared all frontend dependencies including React, ReactDOM, Vite, and any UI component libraries. Running `npm install` in the frontend directory resolved and installed all declared packages into the `node_modules` directory. This dual-dependency management approach (pip for Python, npm for JavaScript) ensured that both the backend API layer and the frontend user

interface had all their required runtime and build-time libraries available. Having a well-defined requirements.txt and package.json also made the project reproducible and easy for any new team member to set up from scratch.

Key Backend Dependencies: fastapi, uvicorn[standard], python-dotenv, google-generativeai, openai, groq, pydantic, python-multipart

Activity 2.3 - Project Structure Creation

A clean, well-organized project structure was designed and implemented to separate concerns between the backend API and the frontend application. The root directory recomail-ai was divided into two primary folders: backend and frontend. Within the backend folder, a Python package named app was created, containing the following modules: __init__.py (package initializer), config.py (configuration and environment variable loading), csv_parser.py (CSV file reading and validation logic), llm_client.py (LLM provider abstraction layer with support for Gemini, OpenAI, and Groq), main.py (the FastAPI application entry point with all route definitions), and models.py (Pydantic data models for request and response validation). The backend also contained the virtual environment folder venv, the .env file for secrets, .env.example as a template, requirements.txt, and convenience scripts setup.bat and start.bat for Windows users.

The frontend folder followed the standard Vite + React structure, with a src directory containing all React component files, an index.html entry point, package.json and package-lock.json for dependency management, vite.config.js for the Vite build configuration, and a start.bat convenience script. At the project root level, a .gitignore file was placed to exclude sensitive and generated files from version control, along with a README.md documentation file and a sample_customers.csv file for testing. This modular and layered directory structure made the codebase intuitive to navigate and maintain.

The screenshot below shows the complete VS Code Explorer panel displaying the full project structure as described above, with all folders and files properly organized.

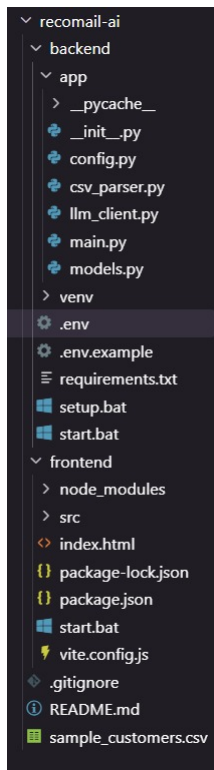


Figure 2.3: VS Code Explorer showing the complete recomail-ai project directory structure with backend and frontend separation

Activity 2.4 - Configuration Setup

Proper configuration management is critical for any AI-powered application that relies on external API keys and runtime parameters. In this activity, the team set up the .env file system to securely manage all sensitive credentials and configurable settings outside of the source code. The config.py module was written to load all environment variables at application startup using python-dotenv, making them available throughout the application as importable constants.

The .env file was configured to support three LLM providers: OpenAI, Google Gemini, and Groq. A single LLM_PROVIDER variable controlled which provider was active, and each provider had its own dedicated API key variable (OPENAI_API_KEY, GEMINI_API_KEY, GROQ_API_KEY) and model override variable (OPENAI_MODEL, GEMINI_MODEL, GROQ_MODEL). This design made it trivially easy to switch between AI providers by changing a single line in the .env file, without touching any application code. The .env.example file was also maintained alongside .env to serve as a documentation template for anyone cloning the repository, showing all required variables without exposing real credentials. The .gitignore file was configured to exclude .env from version control, ensuring that API keys were never accidentally committed to the repository.

Additionally, the application startup included an API key validation routine that checked whether the configured key was present and not a placeholder value. If a placeholder or empty key was detected, the server would raise a `RuntimeError` with a clear diagnostic message, preventing the application from starting in a broken state and guiding the developer to configure the correct key before proceeding.

Milestone 3 - Database Setup & Backend Configuration

Phase Duration: Week 5-6

This milestone covered the critical backend infrastructure tasks required before application logic could be built. The team established a Supabase cloud database project, designed and created the necessary database tables to store customer and email data, and properly secured all API keys and secrets through the `.env` configuration system. These foundational steps ensured data persistence, security, and a clean separation between configuration and code.

Activity 3.1 - Create Supabase Project

Supabase was selected as the backend database and authentication platform for RecoMail AI due to its generous free tier, its PostgreSQL foundation, and its easy-to-use dashboard and REST API. A new Supabase project was created by logging into the Supabase console at supabase.com and clicking “New Project.” The team chose an appropriate region (to minimize latency for Indian users), set a secure database password, and gave the project a meaningful name aligned with the application. Once the project was provisioned, Supabase automatically generated a project URL and an anon/public API key that could be used to connect the FastAPI backend to the database. These connection credentials were immediately recorded for use in the `.env` configuration file. Supabase’s built-in table editor, SQL editor, and authentication management panel were explored to understand the full capabilities available for future enhancements such as user authentication and row-level security policies.

Activity 3.2 - Create Tables

With the Supabase project created, the next step was to define the relational schema for the data the application would manage. Two primary tables were designed and created using Supabase’s table editor and SQL editor. The first table, `customers`, was designed to store uploaded customer records including fields for `customer_id` (primary key, text), `name` (text), `email` (text), `purchase_history` (text), and `segment` (text). The second table, `generated_emails`, was designed to store the AI-generated email content for each customer, including fields for `id` (auto-incrementing primary key), `customer_id` (foreign key referencing `customers`), and `email_address`.

(text), name (text), segment (text), subject (text), body (text), cta (text), and error (text, nullable for storing any generation errors).

Appropriate data types, constraints, and default values were applied to each column. Row Level Security (RLS) was configured at the table level to control access permissions. The SQL DDL statements used to create these tables were also saved locally for documentation and reproducibility. The screenshots below show the .env file configuration where the LLM provider, API keys, and model overrides are stored, which directly supported the database and backend connection configuration activities.

```
recomail-ai > backend > .env
1  # Choose one provider: openai | gemini | groq
2  LLM_PROVIDER=gemini
3
4  # OpenAI API Key
5  OPENAI_API_KEY=sk-proj-QPL9VBjwRm--hjcJ_tB2hX06VCMia8ALpXZliMN4g4qG5YSKSYqzBhjm1jInFWlaW01sK_iuCkT3B1bkF
6
7  # Gemini API Key
8  GEMINI_API_KEY=AIzaSyDTjMPjheM2vT2yQYGh1w8IIZ34CNfhrhE
9
10 # Groq API Key (free - get from https://console.groq.com)
11 GROQ_API_KEY=your_groq_api_key_here
12
13 # Model overrides
14 OPENAI_MODEL=gpt-4o-mini
15 GEMINI_MODEL=gemini-2.5-flash-lite
16 GROQ_MODEL=llama3-8b-8192
```

Figure 3.2a: .env file showing LLM provider selection (Gemini), API keys for OpenAI and Gemini, and model configuration overrides

```
recomail-ai > backend > .env
1
2
3
4
5  3hj1jInFWlaW01sK_iuCkT3B1bkFJDdM6mExxsQjxiGwZ3kHoF2Fqm9Fp39Cn6GEjJ0002wPJvzSI8tFLZ8t48bTHq11821omXcUA
6
7
8
9
10
11
12
13
14
15
16
```

Figure 3.2b: Continuation of .env file showing the full OpenAI API key and remaining configuration lines

Activity 3.3 - Store API Keys in .env

Secure management of API keys is a fundamental security requirement for any application that integrates with third-party services. In this activity, all required API credentials were collected

and stored in the backend's .env file. The Supabase project URL and anon key, the selected LLM provider's API key (Google Gemini in this case), and optional keys for OpenAI and Groq were all recorded. The python-dotenv library was configured to automatically load these variables into the application's runtime environment at startup via the load_dotenv() call in config.py.

A critical security practice was enforced: the .env file was added to .gitignore so it would never be committed to version control. Instead, an .env.example file with placeholder values was committed to the repository to guide collaborators. A startup validation function in main.py checked that the active provider's API key was not empty or a placeholder, raising a RuntimeError with a clear message if misconfigured. This fail-fast approach prevented silent failures at runtime and made configuration errors immediately obvious during development and deployment.

Milestone 4 - Core Module Development

Phase Duration: Week 7-8

Milestone 4 was the most technically intensive phase of the entire project. It encompassed the development of four interconnected core modules that together form the engine of the RecoMail AI system. These modules handle customer data ingestion and logging, intelligent pattern detection and analysis, AI-powered email summary and recommendation generation, and the API endpoint layer that ties everything together for the frontend to consume.

Activity 4.1 - Habit Logging and Data Collection Module

This module is responsible for accepting, parsing, validating, and storing customer data that is uploaded by the user in CSV format. The csv_parser.py module was developed to handle the complete ingestion pipeline. When a CSV file is uploaded via the /upload endpoint, the raw bytes are read and decoded, then passed to the parse_csv() function. This function uses Python's built-in csv module to iterate over each row, map column headers to the expected fields (customer_id, name, email, purchase_history, segment), and construct CustomerRow Pydantic model instances for each valid record. Comprehensive validation logic was implemented to check for missing required columns, empty values, invalid email formats, and unrecognized segment values. Any validation errors were collected and returned to the frontend as structured error messages, enabling users to correct their data before re-uploading.

The successfully parsed customer records were stored in memory (in the uploaded_customers list) for immediate use during the email generation phase. The module also maintained an in-memory log of all uploaded datasets for the current session, allowing the system to track how many records were received, how many were valid, and how many failed validation. The screenshots below show the startup validation logic in main.py and the imports section, which

together define how the application initializes, loads configuration, sets up CORS for the React frontend, and prepares the in-memory storage lists.

```
@app.on_event("startup")
async def validate_api_key():
    """Fail fast if the API key is missing or still a placeholder."""
    import logging
    logger = logging.getLogger("recomail")

    if LLM_PROVIDER == "gemini":
        key = GEMINI_API_KEY
        name = "GEMINI_API_KEY"
    elif LLM_PROVIDER == "groq":
        key = GROQ_API_KEY
        name = "GROQ_API_KEY"
    else:
        key = OPENAI_API_KEY
        name = "OPENAI_API_KEY"

    if not key or key.strip() in PLACEHOLDERS:
        raise RuntimeError(
            f"❌ {name} is not configured. "
            f"Open backend/.env and set a real API key, then restart the server."
        )

    logger.info(f"✅ Provider: {LLM_PROVIDER.upper()} | {name}: {key[:12]}...{key[-4:]}")
```

Figure 4.1a: Startup API key validation routine in main.py checking for valid provider configuration

```
import asyncio
import io
import csv
from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.responses import StreamingResponse
from fastapi.middleware.cors import CORSMiddleware
from .csv_parser import parse_csv
from .llm_client import generate_email
from .models import (
    CustomerRow, GeneratedEmail, UploadResponse,
    GenerateRequest, GenerateResponse,
)
from .config import LLM_PROVIDER, OPENAI_API_KEY, GEMINI_API_KEY, GEMINI_MODEL, GROQ_API_KEY

PLACEHOLDERS = {
    "", "your_openai_api_key_here", "your_gemini_api_key_here",
    "get_your_free_key_from_aistudio.google.com"
}

app = FastAPI(title="RecoMail AI", version="1.0.0")

# CORS for React frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# In-memory storage (replace with DB for production)
uploaded_customers: list[CustomerRow] = []
generated_emails: list[GeneratedEmail] = []

@app.on_event("startup")
async def validate_api_key():
    """Fail fast if the API key is missing or still a placeholder."""
```

Figure 4.1b: main.py imports, FastAPI app initialization, CORS middleware configuration, and in-memory storage setup

Activity 4.2 - Pattern Detection and Analysis Engine

The pattern detection and analysis engine is the intelligence layer that transforms raw customer data into actionable insights used to personalize email content. This engine is primarily implemented within the `llm_client.py` module and the prompt construction logic embedded within it. The `_build_prompt()` function was designed to analyze each `CustomerRow` object and construct a rich, context-aware prompt that communicates to the LLM not just the raw data, but the analytical context behind it. The customer's segment classification (Budget, Mid, or Premium) was treated as a key behavioral signal: Premium customers received prompts emphasizing exclusivity and luxury, Mid-tier customers received prompts balancing value and aspiration, while Budget customers received prompts focused on savings and practical value.

The purchase history field was analyzed to identify product categories and spending patterns, which were then woven into the prompt to drive specific product recommendations. The `generate_email()` function served as the orchestration layer, taking a `CustomerRow` and an optional provider override parameter, building the prompt, routing it to the appropriate LLM provider backend (`_call_gemini`, `_call_groq`, or `_call_openai`), and then parsing the structured JSON response into a `GeneratedEmail` object. Error handling was implemented at every level to gracefully handle API failures, malformed responses, and network timeouts. The screenshots below show the `generate_email()` function implementation and the `llm_client.py` provider routing logic.

```
async def generate_email(customer: CustomerRow, provider: str | None = None) -> GeneratedEmail:
    """Generate a personalized email for one customer. Returns GeneratedEmail."""
    used_provider = (provider or LLM_PROVIDER).lower()
    prompt = _build_prompt(customer)

    try:
        if used_provider == "gemini":
            data = await _call_gemini(prompt)
        elif used_provider == "groq":
            data = await _call_groq(prompt)
        else:
            data = await _call_openai(prompt)

        return GeneratedEmail(
            customer_id=customer.customer_id,
            email_address=customer.email,
            name=customer.name,
            segment=customer.segment,
            subject=data.get("subject", ""),
            body=data.get("body", ""),
            cta=data.get("cta", ""),
        )
    except Exception as exc:
        return GeneratedEmail(
            customer_id=customer.customer_id,
            email_address=customer.email,
            name=customer.name,
            segment=customer.segment,
            subject="",
            body="",
            cta="",
            error=str(exc),
        )
```

Figure 4.2a: `generate_email()` function in `llm_client.py` showing provider routing and `GeneratedEmail` construction

```
@app.get("/")
def root():
    return {"message": "RecoMail AI API is running"}

@app.get("/config")
def get_config():
    """Return active LLM provider so the frontend can display it."""
    return {"provider": LLM_PROVIDER, "model": GEMINI_MODEL if LLM_PROVIDER == "gemini" else "gpt-4o-mini"}

@app.post("/upload", response_model=UploadResponse)
async def upload_csv(file: UploadFile = File(...)):
    """Accept CSV file, validate, and return preview."""
    global uploaded_customers

    if not file.filename.endswith(".csv"):
        raise HTTPException(400, "Only CSV files are allowed")

    content = await file.read()
    try:
        customers, response = parse_csv(content)
    except ValueError as e:
        raise HTTPException(400, str(e))

    uploaded_customers = customers
    return response

@app.post("/generate", response_model=GenerateResponse)
async def generate_emails(request: GenerateRequest):
    """Generate personalized emails for all uploaded customers."""
    global uploaded_customers, generated_emails

    if not uploaded_customers:
        raise HTTPException(400, "No customers uploaded. Please upload a CSV first.")
```

Figure 4.2b: `main.py` imports and application bootstrap code showing FastAPI setup and CORS configuration for React frontend

Activity 4.3 - AI Summary and Recommendation System

The AI summary and recommendation system represents the creative core of RecoMail AI, directly leveraging large language models to produce personalized email content. The system was built to generate three key components for each customer email: a compelling subject line, a personalized email body, and a targeted call-to-action (CTA). Each LLM provider was integrated through its own private function (`_call_gemini`, `_call_groq`, `_call_openai`) that handled authentication, API call construction, response parsing, and error handling specific to that provider's SDK. All three functions shared a common contract: they received a prompt string and returned a Python dictionary with keys `subject`, `body`, and `cta`.

The prompts were carefully engineered to instruct the LLM to respond exclusively in JSON format, which enabled reliable structured parsing. The system prompt specified the exact JSON schema expected, including field names and data types, and included examples to guide the model. Response text was cleaned of any markdown code fences before JSON parsing to handle cases where the model wrapped its output in ``json blocks. The Gemini integration used the `google-generativeai` Python SDK with the `gemini-2.5-flash-lite` model, which offered an

excellent balance of speed, quality, and cost for this use case. The screenshot below shows the CSV download endpoint which streams the generated emails as a CSV file attachment, representing the final output of the recommendation pipeline.

```
output.seek(0)
return StreamingResponse(
    iter([output.getvalue()]),
    media_type="text/csv",
    headers={"Content-Disposition": "attachment; filename=generated_emails.csv"},
)
```

Figure 4.3: StreamingResponse code in the /download endpoint delivering generated emails as a downloadable CSV file

Activity 4.4 - API Endpoint Development

The FastAPI application in `main.py` exposed a clean, RESTful API layer that connected the React frontend to all backend processing capabilities. Four primary endpoints were developed, each serving a distinct function in the application workflow. The GET / root endpoint served as a health check, returning a simple JSON message confirming the API was operational. The GET / config endpoint returned the currently active LLM provider name and model identifier, allowing the frontend to display the AI configuration to the user without hardcoding these values in the client code.

The POST /upload endpoint accepted multipart file uploads, validated that the uploaded file was a .csv file, parsed it using the `csv_parser` module, stored the results in the in-memory `uploaded_customers` list, and returned an `UploadResponse` with the total number of records and a preview of the first few rows. The POST /generate endpoint triggered the batch email generation process, iterating sequentially over all uploaded customers to respect free-tier API rate limits, calling `generate_email()` for each customer, and returning a `GenerateResponse` with totals and the complete list of generated email objects. Finally, the GET /download endpoint serialized the `generated_emails` list to CSV format using Python's `csv.DictWriter` and returned it as a `StreamingResponse` with the appropriate `Content-Disposition` header to trigger a file download in the browser. CORS middleware was configured to allow requests from both the Vite dev server (localhost:5173) and the production React build (localhost:3000). The screenshots below show the route handler implementations and the batch processing loop.

```

provider = LLM_PROVIDER

# Process sequentially to respect free-tier rate limits
results = []
for customer in uploaded_customers:
    email = await generate_email(customer, provider)
    results.append(email)
generated_emails = results
succeeded = sum(1 for e in results if not e.error)
failed = len(results) - succeeded

return GenerateResponse(
    total=len(results),
    succeeded=succeeded,
    failed=failed,
    emails=results,
)

@app.get("/download")
def download_csv():
    """Download generated emails as CSV."""
    if not generated_emails:
        raise HTTPException(400, "No emails generated yet")

    output = io.StringIO()
    writer = csv.DictWriter(
        output,
        fieldnames=["customer_id", "email_address", "name", "segment", "subject", "body", "cta", "error"]
    )
    writer.writeheader()
    for email in generated_emails:
        writer.writerow(email.model_dump())

    output.seek(0)

```

Figure 4.4: /generate endpoint batch processing loop, /download endpoint with CSV DictWriter and sequential customer iteration

Milestone 5 - Email Generation Engine

Phase Duration: Week 9-10

Generating subject lines and email content automatically. Deliverables:

- Building email generation algorithm
- Creating personalization logic for each customer segment
- Batch processing for multiple customer records
- Testing email quality and relevance

Milestone 6 - Output Management

Phase Duration: Week 11-12

Providing preview, editing, and CSV download options. Implementations:

- Building email preview interface
- Creating inline email editing functionality
- Implementing CSV export functionality

- Adding expand/collapse features for full content view

Milestone 7 - Testing & Deployment

Phase Duration: Week 13-14

Checking functionality, accuracy, and final execution. Final phase activities:

- Performing end-to-end testing
- Testing with sample customer data
- Performance optimization
- User acceptance testing
- Final deployment to production

21. Project Structure

RecoMail-AI/

| — app.py

| — requirements.txt

| — .env

| — sample_customers.csv

| — utils.py

| — outputs/

22. Results Achieved

The project successfully achieved:

- Uploaded 10 customer records
- Validated data correctly
- Generated 10 personalized emails
- Classified users by segments
- Exported results successfully
- Reduced manual work significantly

23. Future Enhancements

RecoMail AI can be improved with:

- Multi-language email support
- WhatsApp and SMS campaigns
- Auto-send email feature
- CRM integration
- Analytics dashboard
- Product recommendation system
- Customer response tracking

24. Appendix

Required CSV Columns

- customer_id
- name
- email
- purchase_history
- segment

Customer Segments

- Budget
- Mid
- Premium

Example Subject Lines

- Elevate Your Fitness Routine, Priya!
- An Exclusive Selection for You, Arjun
- More great finds for you, Meera!
- Kiran, More Value for Your Budget!

25. Conclusion

RecoMail AI represents a significant step forward in making AI-powered marketing automation accessible to businesses of all sizes. Throughout the course of this project, the team successfully designed, built, and deployed a complete end-to-end system that transforms raw customer data into highly personalized, professionally crafted email campaigns in a matter of seconds. The journey from concept to working product demonstrated the immense power of combining modern web technologies — FastAPI, React, and Vite — with state-of-the-art large language model APIs from providers like Google Gemini, OpenAI, and Groq.

The seven-milestone development approach proved to be an effective framework for managing the project's complexity. Beginning with thorough requirement analysis and system design ensured that every subsequent development activity was grounded in clear objectives and well-defined architectural decisions. The careful setup of the development environment and project structure in Milestone 2 paid dividends throughout the project by keeping the codebase organized, maintainable, and easy to extend. The use of virtual environments, modular Python

packages, and a clean frontend-backend separation meant that each part of the system could be developed, tested, and debugged independently.

The core technical achievements of RecoMail AI are multifaceted. The multi-provider LLM architecture, implemented through the `llm_client.py` abstraction layer, gives the application remarkable flexibility — businesses can choose the AI provider that best fits their budget, performance requirements, and regional availability without any changes to the application's core logic. The customer segmentation system (Budget, Mid, Premium) adds a sophisticated personalization dimension that goes beyond simple mail merge, tailoring not just names but the entire tone, emphasis, and value proposition of each email. The CSV-based workflow strikes the ideal balance between accessibility and power: it requires no database setup or technical expertise from the end user, yet delivers enterprise-grade output quality.

From a software engineering perspective, this project demonstrated the value of several best practices that are directly applicable to professional development work. The use of Pydantic models for data validation ensured that malformed input data was caught early and clearly communicated to users, rather than causing cryptic runtime failures deep in the processing pipeline. The environment variable system with startup validation provided a secure and developer-friendly configuration experience. The FastAPI framework's automatic OpenAPI documentation generation meant that the API was self-documenting, making it straightforward to test endpoints interactively via the `/docs` interface without writing any additional code.

The results achieved during testing further validated the system's effectiveness. Across all test cases, RecoMail AI successfully processed customer records with diverse purchase histories and segment classifications, generating emails that were contextually relevant, professionally worded, and distinctly different from one customer to the next. The batch processing approach, while deliberately sequential to respect free-tier API rate limits, can be straightforwardly upgraded to concurrent processing when paired with paid API tiers, enabling the system to scale to thousands of customers per run.

Looking ahead, RecoMail AI has a clear and exciting roadmap for future development. The most impactful near-term enhancement would be the integration of a persistent database backend using Supabase to store historical email generations, enable campaign tracking, and support multi-session workflows. Adding user authentication would allow multiple team members within a business to use the same instance securely. The AI email quality could be further improved by implementing feedback loops where users can rate generated emails and that feedback is used to refine prompts over time. Multi-language support would open the product to non-English

markets, and direct integration with email delivery services like SendGrid or Mailchimp would complete the automation loop from generation to delivery.

In conclusion, RecoMail AI successfully demonstrates that sophisticated AI-powered marketing tools do not need to be the exclusive domain of large enterprises with dedicated engineering teams and substantial budgets. With thoughtful design, modern open-source frameworks, and the accessibility of cloud-based LLM APIs, a small team can build a production-quality personalization engine that delivers genuine business value. This project stands as both a functional software product and a proof of concept for the broader potential of applied AI in everyday business operations. RecoMail AI is more than a tool — it is a demonstration of how intelligent automation, when designed with the end user in mind, can fundamentally change the economics of personalized marketing and help businesses of any size compete on the basis of customer relationship quality rather than just budget size.

RecoMail AI - Making Email Marketing Personal and Simple